- Renzo Zegers
- Srdan Lunic
- Naman Hegde
- Rytis Jokūbas Minkevicius
- Osman Allahverdi
- Sjoerd Karel van Oostveen

Report on ERD/Class/Class Design Alignment and Use-Cases

The solution architecture stays consistent with a three-layered approach. Evaluating the entity relationship diagram (ERD) defines the database physical schema with integrity enforcement; the Java class diagram correlates to the domain, service and resource layers that process those tables; the use-case specifications are the project requirements set forth by the project stakeholders that ultimately govern schema and code resolutions. This sprint encompassed "must-have" elements for minimumt—authentication, system registration with risk scoring, nightly CVE imports/matches with in-app notifications and status updates with audit trail and registration and dashboard reporting, as well as "should-have" elements that will be added later for more customizability—variable user/role/department management, CVE search.

Authentication and UserAccount Management

Authentication and UserAccount management reflect in the ERD table that indicates a UserAccount table exists and columns UUID id, username, email, passwordHash, role, departmentId, isActive, createdAt directly match the UserAccount model class. A subsequent Role table derived from our UserRole enumeration maintains validation that only permitted roles (admin, security_officer, system_owner, technical_expert) are assigned. Authentication is done via stateless session with JWT. Endpoints POST /auth/login, POST /auth/logout, GET /auth/validate are situated in AuthResource and AuthService classes; the AuthFilter class intercepts requests to ensure token validation. By persisting role & userId in the payload of the JWT, all resource methods are granted access quicker than if @RolesAllowed was used; concurrently, the ERD foreign-key relation between UserAccount.role_id and Role.id maintains referential integrity at the database level while class diagrams one-to-one correlation between model fields and table columns support impedance loss reduction between application classes and database.

System Registration and Risk Input

Support for the requirement "As a System Owner, I want to register a system and enter risk scores" comes from two tables in our ERD for System—for static metadata (name, vendor, description)—and SystemImplementation—for implementation-specific data (department, data type classification flag, risk level). The join table SystemOwner allows for multiple system ownership down the line. In code, the ITSystem model possesses all fields necessary to create persistence via the SystemDAO and SystemService. The SystemService createSystem(ITSystem) method will assess internetFacing, dataClassification, and criticalityLevel fields according to domain validation requirements to ultimately create a riskScore after persisting the super-entity via SystemDAO. In SystemResource, the single POST /systems expects this method while GET /systems and GET /systems/owner/{ownerId} allow appropriate roles to acquire data and authorize responses. The ERD normalization implies metadata about systems is maintained across

different entities consistently without redundancy so an arguably uniquely implemented system is scored only once.

Nightly CVE Import and System Matching

To satisfy the use case "Nightly CVE Import," a Vulnerability table must be present to hold imported attributes—cve_id, description, severity, cvss_score, published_date, imported_at—and a VulnerabilityMatch table to indicate when an imported CVE matched an installed implementation. The VulnerabilityService is called by Scheduler for nightly implementation of NIST NVD API; it obtains/ingests tuples from the vulnerability table into new Vulnerability objects via JSON payload parsing. It checks whether an input cve_id already exists—vulnerabilityDAO.existsByCveId(). When it does not exist/it is new—vulnerabilityDao.persist(new Vulnerability)—it generates—and if applicable—AI-generated scores—persisted in VulnerabilityMatch in association with each Implementation on ITSystem—which holds as composite keys—the subsequent dependency arrows from VulnerabilityService to VulnerabilityDAO, SystemDAO and NotificationService will represent this flow of functionality on the class diagram. The match records cannot duplicate; therefore, a composite unique constraint on (vulnerability_id, system_implementation_id) avoids redundancy of multiple match records per implementation.

In-App Notification to Display that New CVEs are Matched

To satisfy "As a System Owner, I want in-app notifications when new CVEs match my systems," a Notification table was established as related to UserAccount (user_id) and VulnerabilityMatch (match_id). The NotificationService.createVulnerabilityNotification() method will create a notification entry after an entry is created with a user-friendly message to the System Owner. The entry is saved with NotificationDAO.create(). The NotificationResource exposes GET /notifications/user/{userId} and PUT /notifications/{id}/read to show and update status, respectively. In the class diagram, the one-to-many from VulnerabilityMatch to Notification represents how "each match may create multiple notifications." The one-to-many from UserAccount to Notification represents how "each user may receive many notifications." This features every critical occurrence brought to the forefront at the appropriate time, emphasizing the need for communication and tracking of unread notifications.

Change CVE Status When Received with Audit Trail

For "As a System Owner, I want to update CVE statuses with an audit trail," two new tables were synthesized. A VulnerabilityAssessment table holds assessments based on the most current status determined by a change by a system owner. It's related to VulnerabilityMatch with fields status, explanation, flag for escalation, due_date, and completed_date. An AssessmentHistory table includes fields old_status, new_status, change_reason, and timestamp related to VulnerabilityAssessment. The class structure will include the relevant model classes as well as hypothetical DAO/service/resource layers (e.g., AssessmentService and AssessmentResource). This gives a PUT /assessments/{matchId}/status and a rudimentary change of status with an automatic appending of history table entry. The ERD makes foreign-key constraints exist so an assessment history can never be without a parent VulnerabilityAssessment entry. The class diagram includes a one-to-many assessment from VulnerabilityAssessment to AssessmentHistory and appropriately titled associations to capture the audit relationship.

Listen to Need for Dashboard Reports and Create ReportLog

Finally, "As a Security Officer, I want a dashboard of open versus resolved CVEs," is accomplished by implementing the ReportLog table to contain all reports run. Derived fields include generated (who ran it), time frame (date_range), visualizations, filters (file_format), and generated_at (when the report was finally run). Coding includes the ReportService and ReportResource containing POST /reports/dashboard which counts up how many are open/resolved via multiple DAO queries, returns a JSON payload back to the client, and creates an entry via ReportLogDAO.create(). The class diagram shows ReportResource dependent upon ReportService which is dependent upon ReportLogDAO at its middle tier; this shows an end-to-end reporting process. The ERD ensures every report run can be tracked in compliance audits over time.

Must-Haves/Should-Have Logic and Future Extend-Ability

While much of the back end fulfills must-have considerations, aspects like "should-haves" such as full-feature CRUD of user/role/department management and extended CVE search by keyword are layered in parallel. The ERD already provides Role, Department, Organization, and UserDepartment tables; the class diagram extends resources (UserResource, RoleResource, DepartmentResource, VulnerabilityResource) and services (UserService, RoleService, DepartmentService, VulnerabilityService) to include POST /users, GET /roles, PUT /departments/{id}, GET /vulnerabilities/q={term}. There are DAO classes responsible for each (UserAccountDAO, RoleDAO, DepartmentDAO, VulnerabilityDAO) which implement said database operations required. Therefore, by putting these features in a separate module "should-have" within my UML, I've kept a clear line for our—our must-have back end can be deployed/validated without relying upon these administrative/search UI layers that could be plugged in later without changing schemas.

Normalization of the Database Design

The database design is in the form of relational normalization because we seek to avoid redundancy and ensure integration and maintenance is as easy as possible. The primary key of each table (a UUID) signifies that each entry in that table is unique (First Normal Form). Each table's non-key dependent attributes depend on the key and nothing else (Second Normal Form); for example, the SystemImplementation table breaks out system metadata—name, vendor, description—into the System table so that if a system's primary characteristics change, there is only one place to do it. Each dependency as a foreign key (SystemImplementation→System, VulnerabilityMatch→Vulnerability and SystemImplementation, UserAccount→Department) ensures that the child table does not hold data that does not belong so that there is no insertion, update, or deletion anomaly to worry about happening in multiple areas.

Furthermore, we avoid transitive dependencies (Third Normal Form) with join tables (UserDepartment, SystemOwner, VulnerabilityMatch) where the meaning of an attribute rests on its relationship to another set rather than being understood by what it is known. For example, a user can belong to many departments, so there is no need to redundantly state this in the UserAccount table in department fields; it can live independently in UserDepartment. Finally, we use enumerated values and check constraints for roles, risk levels, assessment status, notification priority to guarantee domain integrity without needing character or numeric searches in multiple other tables. Ultimately, this composite

decomposition allows for future expansion—if threaded comments or CSV imports need to be added down the line, only new tables with a focused theme will need to be made—and assists with reliable and consistent execution of all use cases.

Conclusion

Every piece in our architecture ties back to a use case from stakeholder requirements to ERD and class design in Java to RESTful implementation. The ERD features all requisite multiplicities and foreign keys for integrity and normalization of the data. The class diagram is a one-to-one correspondence of the ERD for model classes and extended for service classes due to requisite business logic for the system and exposed resource classes for RESTful APIs due to the documented RESTful requirements. Use cases that are should and could have are inherently planned for within the application. For example, user_id will correspond to the owner of a notification, assessment, report log, and deployed system so that all assignments correspond appropriately in the UML. Musts are provided first with scaffolding set for shoulds to create a dynamic, safe, extensible system for ongoing CVE-related management.